Day 14 Activity: Get Git working (weekend setup)

Friday, June 5, 2026 · due before class Monday, June 8

Your Name Here

This is your weekend task, not an in-class coding activity. By **Monday June 8** your project team needs a shared GitHub repo that **every member can push to.**

Work through the steps in order. Each links to the matching chapter of **Happy Git with R** (Jenny Bryan) — read its instructions; this page is the checklist. **Expect a snag or two.** Part 5 is the troubleshooting playbook, and office hours exist for exactly this.

> **Note**
>
> We use **RStudio's built-in Git integration** with **HTTPS + a personal access token (PAT)**. That matches *DC* Ch. 9 and Happy Git, and keeps everything inside the RStudio you've used since week 1.

Part 1 — Accounts and installs (solo)

- ☐ **Register a GitHub account** — happygitwithr.com/github-acct. Pick a username that's professional-ish; you may reuse it for years.
- ☐ **R and RStudio up to date** — install-r-rstudio. You set these up in week 1; just confirm they're current.
- ☐ **Install Git** — install-git. (Windows: Git for Windows. macOS: easiest is to run `git --version` in the Terminal and accept the install prompt.)
- ☐ **Introduce yourself to Git** — hello-git. Set your name and the email tied to your GitHub account. Easiest from the R console:

```r
# install.packages("usethis")   # once, if you don't have it
library(usethis)
use_git_config(
  user.name  = "Your Name",          # your name, doesn't have to be your username
  user.email = "you@example.com"      # the SAME email as your GitHub account
)
```

Part 2 — Connect RStudio, Git, and GitHub (solo)

This is the finicky part. Go slowly.

- ☐ **Create a personal access token (PAT)** — https-pat. From the R console:

```r
# install.packages(c("usethis", "gitcreds"))   # once, if needed
usethis::create_github_token()   # opens GitHub; generate the token, COPY it
gitcreds::gitcreds_set()         # paste the token when prompted
```

> ⚠ Warning
>
> Treat the token like a password. Copy it the moment you create it, GitHub won't show it again. If you lose it, just make a new one.

- ☐ **Confirm the connection works** — push-pull-github.
- ☐ **Confirm RStudio sees Git** — rstudio-git-github and rstudio-see-git. Check *Tools* → *Global Options* → *Git/SVN* shows a Git executable.

Part 3 — Prove it works with a test repo (solo)

Follow new-github-first:

- ☐ On **GitHub**, create a new repository (check "Add a README").
- ☐ Copy its URL (the green **Code** button → HTTPS).
- ☐ In **RStudio**: *File* → *New Project* → *Version Control* → *Git*, paste the URL, create.
- ☐ Open `README.md`, add a line like "Test repo for STAT 184", and **save**.

- ☐ In the **Git tab** (top-right pane): tick the box to **stage** `README.md` → click **Commit** → write a message → **Push** (up arrow).
- ☐ Refresh the repo page on GitHub. **Your line is there.**

If your change shows up on GitHub, your setup is done. The rest of the term is just repeating this loop.

Part 4 — The team repo (together, async over the weekend)

This is the graded deliverable. Coordinate over your team channel.

One person (the repo owner):

- ☐ Create the team repo on GitHub (with a README). Name it something like `stat184-project-teamNN`.
- ☐ *Settings* → *Collaborators* → add each teammate by GitHub username.

Everyone else:

- ☐ Accept the email/GitHub invitation.
- ☐ Clone the repo into RStudio (*New Project* → *Version Control* → *Git*, paste the URL).

Each member, including the owner:

- ☐ **Pull** first (down arrow) to get everyone's latest.
- ☐ Add your name on its own line in the README, save.
- ☐ Stage → Commit (message like "Add Jordan to README") → **Push**.
- ☐ **Verify:** on GitHub, the README lists **every** member's name, and the repo's commit history shows **at least one commit per person.**

> Tip
>
> If two of you edit the README at the same time, you may hit a **merge conflict**, that's fine, it's good practice. See Part 5, and remember the rule: **pull before you push.**

Part 5 — When it goes wrong (troubleshooting)

You will hit at least one of these. None of them mean you broke anything.

Table 1: Common setup snags.

Symptom	What's happening	What to do
Push rejected	A teammate pushed since your last pull	**Pull**, then push again
Merge conflict	Two people changed the same lines	Open the file, keep the right lines, remove the `<<<<`/`====`/`>>>>` markers, commit
Changes don't appear in Git tab	Wrong RStudio **Project** is open	Switch to the Project tied to that repo
"File too large" (>50 MB)	A big data file got staged	Don't commit it; add it to `.gitignore`
Auth fails on push	PAT missing/expired	Redo `gitcreds::gitcreds_set()` (Part 2)

Still stuck after 15 minutes? Two reliable resources:

- Happy Git's **troubleshooting** chapter.
- The "**Burn it all down**" move: delete the broken local folder and **re-clone** a fresh copy from GitHub. Your pushed work is safe on the remote.

And: bring it to office hours. Setup problems are quick to fix in person and miserable to fix alone at midnight.

Turn in

Submit on Canvas:

1. The **URL of your team repo**.
2. Confirmation that **all members have pushed** (the easiest proof is a screenshot of the repo's commit history or contributors list).

Due before class Monday, June 8. Bring your configured laptop, we'll do a collaborative round-trip and a deliberate merge conflict together.